Entrega 2: Integración de Datos No Estructurados (NoSQL)

Solsticio: estructura de datos en MongoDB y justificación de la persistencia políglota

Presentado por:

Joan Sebastián Lara Fuenmayor

Miguel Angel Marin Baracaldo

Docente:

Leandro Pájaro Fuentes

Fundación Universitaria Konrad Lorenz

Facultad de Matemáticas e Ingenierías

Ingeniería de Sistemas

Nuevas Tecnologías de Desarrollo

Bogotá D.C., 4 de noviembre de 2026

Índice

1. Introducción . . . 3

2. Objetivos . . . 3

3. Identificación de los datos no estructurados . . . 3

3.1 Bitácora de actividad de usuarios . . . 4

3.2 Escaneos de códigos QR . . . 4

3.3 Reseñas de productos . . . 4

3.4 Ficha técnica con atributos variables . . . 5

4. Elección de la base de datos NoSQL . . . 5

5. Arquitectura de persistencia políglota . . . 5

5.1 Reglas de consistencia entre ambas bases . . . 6

6. Modelo de datos . . . 6

7. Estructura de las colecciones . . . 8

7.1 actividad_usuario . . . 8

7.2 escaneos_qr . . . 9

7.3 resenas_producto . . . 9

7.4 fichas_producto . . . 10

8. Lógica de negocio que vincula ambos tipos de datos . . . 11

9. Operaciones CRUD implementadas . . . 12

10. Validación y seguridad . . . 12

11. Configuración y conexión . . . 12

12. Interfaz web y pruebas automáticas . . . 13

13. Evidencias de ejecución . . . 14

14. Conclusiones . . . 17

Referencias . . . 18

1. Introducción

Solsticio es una API REST en Flask que centraliza el inventario y las ventas de una tienda de ropa con varias sedes en Bogotá. En la Entrega 1 todo el sistema se modeló en MySQL 8.0: 23 tablas normalizadas para catálogo (productos, variantes por talla y color), existencias por sede, movimientos de inventario, traslados, pedidos web y de mostrador, pagos, facturación electrónica y devoluciones.

Ese modelo relacional es el adecuado para el núcleo transaccional: vender exige descontar stock, registrar el movimiento y crear el pedido en una sola transacción, con llaves foráneas que impidan inconsistencias. Sin embargo, al crecer la aplicación aparecieron datos que no encajan en tablas rígidas: eventos que llegan en cada petición, opiniones de clientes con contenido variable y fichas técnicas cuyos atributos dependen del tipo de prenda. Esta entrega incorpora **MongoDB** (versión 7 o superior; probado con 7.0 y 9.0) como base documental para esos datos y deja a MySQL lo que necesita consistencia fuerte.

Este documento explica qué datos se llevaron a MongoDB y por qué, describe la estructura de cada colección con sus índices, muestra cómo la aplicación combina ambas bases y presenta las evidencias de ejecución.

Repositorio: https://github.com/miguelamarinb-ops/Solsticio. El archivo README explica la instalación, la ejecución y los endpoints; la carpeta *docs/* contiene este documento, los diagramas y las capturas.

2. Objetivos

Objetivo general: integrar una base de datos documental (MongoDB) a la API de Solsticio para manejar la información que no se ajusta al modelo rígido de tablas, sin perder la consistencia transaccional de MySQL.

Objetivos específicos:

- Identificar qué datos del negocio se benefician de un modelo NoSQL y justificar la elección con criterios explícitos.
- Diseñar la estructura de las colecciones, sus campos, índices y la forma en que se enlazan con las tablas de MySQL.
- Implementar la conexión, las operaciones CRUD y al menos un endpoint que combine datos SQL y NoSQL.
- Verificar el funcionamiento con pruebas automáticas y evidencias de ejecución.

3. Identificación de los datos no estructurados

Para decidir qué datos van a cada base se evaluaron cuatro criterios sobre cada tipo de información del negocio:

- **Variabilidad del esquema:** ¿todos los registros tienen los mismos campos?
- **Frecuencia de escritura:** ¿se escribe ocasionalmente o en cada interacción?
- **Necesidad transaccional:** ¿un error deja al negocio en un estado inconsistente (stock negativo, venta sin factura)?
- **Forma de lectura:** ¿se lee como un todo (un documento) o se cruza con muchas tablas?

Dato	Esquema	Escritura	¿ACID?	Lectura	Base elegida
Productos, variantes, precios	Fijo	Baja	Sí	Joins con stock y pedidos	MySQL
Stock y movimientos	Fijo	Media	Sí (crítico)	Por sede y variante	MySQL
Pedidos, pagos, facturas	Fijo	Media	Sí (crítico)	Joins, reportes contables	MySQL
Bitácora de actividad	Variable por acción	Muy alta (cada petición)	No	Últimos N por usuario, agregados	MongoDB
Escaneos de QR	Variable (canal, sede, usuario opcionales)	Muy alta	No	Agregados por variante	MongoDB
Reseñas de clientes	Variable (atributos, fotos, respuestas)	Media	No	Documento completo + agregados	MongoDB
Ficha técnica de la prenda	Muy variable por categoría	Baja	No	Documento completo, filtros por atributo	MongoDB

3.1 Bitácora de actividad de usuarios

Cada petición autenticada (un trabajador que consulta stock, registra un ingreso, cobra un pedido o despacha un traslado) genera un evento. En una tienda con varias sedes y personal trabajando todo el día esto produce miles de registros diarios. Los campos cambian según la acción: un traslado tiene *id_traslado*, una consulta de stock tiene filtros, un error tiene su mensaje. En MySQL habría que elegir entre muchas columnas nulas o un campo JSON sin índices útiles, y cada inserción competiría con las transacciones de venta. Además estos datos pierden valor con el tiempo: basta conservarlos 180 días.

3.2 Escaneos de códigos QR

Cada prenda tiene un QR que el cliente escanea en la tienda o desde la web para ver disponibilidad por sede. Cada lectura es un evento independiente, de solo inserción, cuyo valor está en el agregado: qué referencias generan más interés y por qué canal. Incluso se registran los códigos inexistentes (etiquetas dañadas o falsificadas).

3.3 Reseñas de productos

Una reseña tiene texto libre, una calificación y, opcionalmente, talla comprada, opinión sobre el ajuste, atributos que el cliente quiera calificar (calidad de la tela, si encoge, si se ve igual que en la foto), fotos y las respuestas de la tienda. En un modelo relacional serían al menos cuatro tablas (reseña, atributo_reseña, foto_reseña, respuesta_reseña) que siempre se leen juntas. Como documento, la reseña se lee y escribe completa en una sola operación.

3.4 Ficha técnica con atributos variables

Es el caso típico de catálogo con atributos variables. Una camiseta tiene gramaje, tipo de cuello y manga; un jean tiene tiro, elasticidad, lavado y número de bolsillos; unos tenis tendrían suela y horma. Agregar una columna en MySQL por cada atributo posible deja la mayoría vacías y obliga a migrar el esquema cada vez que llega una categoría nueva; el patrón EAV (entidad-atributo-valor) evita eso pero vuelve muy costosas las consultas. En MongoDB cada ficha guarda solo los atributos que le aplican, con estructuras anidadas (por ejemplo la composición como lista de fibras y porcentajes).

4. Elección de la base de datos NoSQL

Se eligió una base **documental**, MongoDB, frente a las otras familias NoSQL:

Familia	Ejemplo	Por qué no / por qué sí
Clave-valor	Redis	Muy rápida, pero no permite consultar por campos internos (atributos, calificación) ni hacer agregaciones. Útil como caché, no como almacén principal de estos datos.
Columnar	Cassandra	Pensada para volúmenes de escritura muy superiores a los de Solsticio y con consultas definidas de antemano por la llave de partición; su operación es más compleja.
Grafos	Neo4j	Ideal para relaciones (recomendaciones), pero ninguno de los cuatro casos es un problema de grafos.
Documental	**MongoDB**	**Esquema flexible por documento, subdocumentos y arreglos (composición, fotos, respuestas), índices secundarios y TTL, pipeline de agregación para reportes, driver oficial maduro para Python (PyMongo) y opción gratuita en la nube (Atlas) para el despliegue en Render.**

5. Arquitectura de persistencia políglota

La API consulta MySQL para los datos estructurados y MongoDB para los dinámicos o de alta frecuencia. Ambas bases se enlazan por identificador: los documentos guardan *id_usuario*, *id_producto* e *id_variante* de MySQL, y la ficha técnica usa directamente el *id_producto* como *_id*.

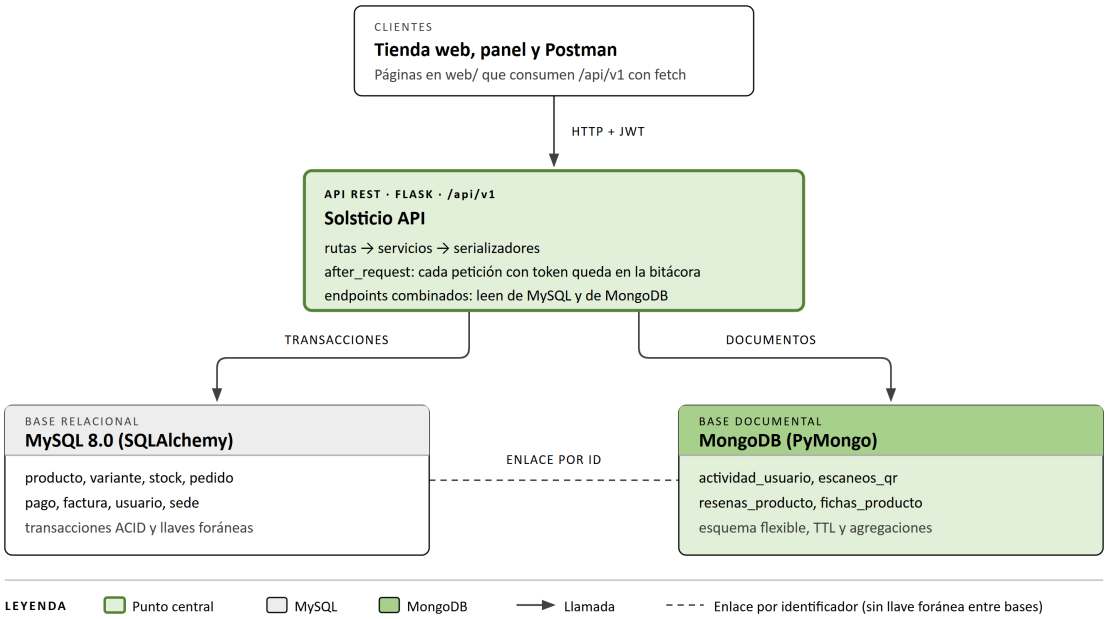


Figura 1. Arquitectura de persistencia políglota: la API decide qué va a cada base.

5.1 Reglas de consistencia entre ambas bases

- **MySQL es la fuente de verdad** de todo lo que tiene valor contable: MongoDB nunca guarda precios ni existencias. Las respuestas combinadas siempre leen precio y stock vigentes de MySQL.
- **Validación cruzada antes de escribir:** antes de crear una ficha o una reseña se comprueba en MySQL que el producto exista y esté activo; si la reseña trae un código de pedido, se verifica que ese pedido esté pagado e incluya el producto.
- **Desnormalización mínima y explícita:** se copian a MongoDB solo datos descriptivos que sirven para filtrar sin ir a MySQL (nombre y categoría en la ficha; talla y color en la reseña), como instantánea.
- **Tolerancia a fallos:** si MongoDB cae, ninguna venta se cancela. El registro de eventos falla en silencio (con aviso en el log), la vista combinada del producto responde con la parte de MySQL y *nosql_disponible: false*, y los endpoints puramente NoSQL responden 503.

6. Modelo de datos

La figura siguiente muestra la estructura completa: a la izquierda las tablas de MySQL que intervienen y a la derecha las cuatro colecciones de MongoDB, con el campo por el que se enlazan y la cardinalidad de cada relación. No hay llaves foráneas entre motores: la integridad la garantiza la API, que valida en MySQL antes de escribir en MongoDB. La colección *fichas_producto* se destaca porque es la que más aprovecha el esquema flexible: su *_id* es el mismo *id_producto* de MySQL (relación 1 a 1) y sus atributos cambian según la prenda.

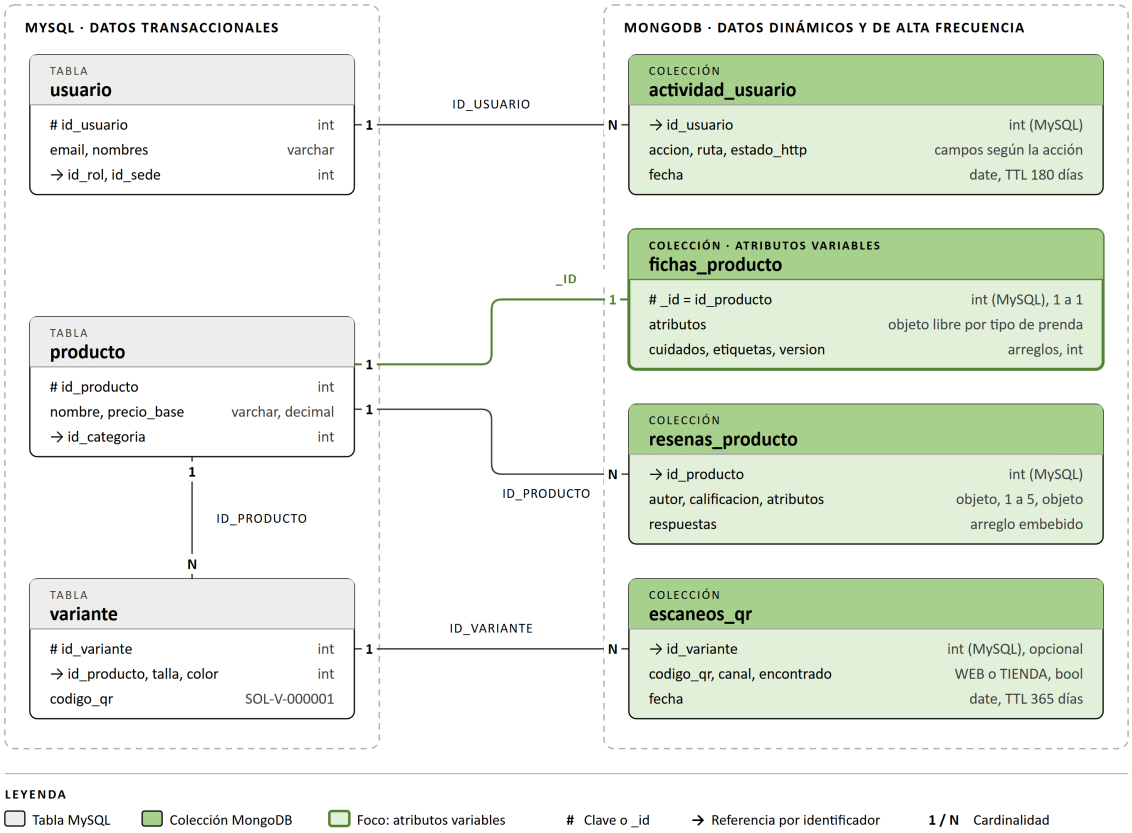


Figura 2. Modelo de datos: tablas de MySQL y colecciones de MongoDB enlazadas por identificador.

7. Estructura de las colecciones

La base documental se llama **solsticio** y tiene cuatro colecciones. Los índices se crean automáticamente al iniciar la API (función *crear_indices* en *api/nosql.py*) y con el comando *flask --app app verificar-mongo*.

7.1 actividad_usuario

Un documento por cada petición hecha con token. Se escribe desde un único punto (*after_request*), sin añadir código a cada endpoint.

Campo	Tipo	Oblig.	Descripción
_id	ObjectId	Sí	Generado por MongoDB (incluye la marca de tiempo de inserción).
id_usuario	int	Sí	Referencia a usuario.id_usuario (MySQL).
rol, id_sede	string, int|null	Sí	Copia del rol y la sede al momento de la acción (histórico).
accion	string	Sí	Nombre de negocio: INICIO_SESION, VENTA_REGISTRADA, TRASLADO_DESPACHADO, ESCANEO_QR...
metodo, ruta, estado_http	string, string, int	Sí	Petición HTTP y su resultado.
exito	bool	Sí	estado_http < 400. Permite filtrar fallos rápidamente.
parametros	object	No	Parámetros de la URL (id_pedido, id_traslado...). Solo existe si la ruta los tiene.
filtros	object	No	Query string usada (sede, fechas, página...).
error	string	No	Mensaje de error devuelto, solo en respuestas 4xx/5xx.
ip, user_agent	string	Sí	Origen de la petición (auditoría).
fecha	date (UTC)	Sí	Momento del evento.

Índices: {id_usuario: 1, fecha: -1} para «últimas N actividades de un usuario»; {accion: 1, fecha: -1} para la bitácora filtrada; TTL sobre *fecha* con *expireAfterSeconds* = 180 días, de modo que MongoDB borra solo los eventos antiguos y la colección no crece sin límite.

```
{
  "_id": ObjectId("6aeb30500dbf00a6fb8fc82b"),
  "id_usuario": 1, "rol": "ADMINISTRADOR", "id_sede": null,
  "accion": "FICHA_GUARDADA", "metodo": "PUT", "ruta": "/api/v1/productos/1/ficha",
  "estado_http": 200, "exito": true,
  "parametros": { "id_producto": 1 },
  "ip": "127.0.0.1", "user_agent": "PostmanRuntime/7.43.0",
  "fecha": ISODate("2026-11-04T13:17:04Z")
}
```

7.2 escaneos_qr

Campo	Tipo	Oblig.	Descripción
codigo_qr	string	Sí	Código leído, exista o no.
encontrado	bool	Sí	Si el código corresponde a una variante activa en MySQL.
id_variante, id_producto	int	No	Solo si se encontró. Referencias a MySQL.
canal	string	Sí	WEB (sin token, cliente) o TIENDA (con token, personal).
id_sede	int	No	Sede donde se escaneó (?sede= o sede del trabajador).
id_usuario	int	No	Solo si el escaneo lo hizo el personal.
ip, user_agent, fecha	string, string, date	Sí	Origen y momento.

Índices: {id_variante: 1, fecha: -1}, {id_producto: 1, fecha: -1} y TTL de 365 días. Los campos opcionales no se guardan como *null*: simplemente no existen en el documento.

7.3 resenas_producto

Campo	Tipo	Oblig.	Descripción
id_producto	int	Sí	Referencia a producto (MySQL).
autor	object {nombre, email}	Sí	El email evita duplicados y nunca se devuelve en endpoints públicos.
calificacion	int 1–5	Sí	
comentario / titulo	string	Sí / No	Máx. 2000 / 120 caracteres.
id_variante, talla, color	int, string, string	No	Variante comprada; talla y color copiados de MySQL.
ajuste_talla	string	No	PEQUENA, EXACTA o GRANDE.
atributos	object	No	Libre, plano, máx. 15 claves: {calidad_tela: 5, encoge: false}.
fotos	array<string>	No	Hasta 5 URLs.
codigo_pedido, compra_verificada	string, bool	No / Sí	Verificada contra pedido y detalle_pedido en MySQL.
estado	string	Sí	PUBLICADA u OCULTA (moderación).
util	int	Sí	Votos de utilidad, incrementados con $inc (atómico).
respuestas	array<object>	Sí	Respuestas de la tienda **embebidas**: {id_usuario, nombre, texto, fecha}.
fecha, fecha_moderacion	date	Sí / No	

Embebido vs. referencia: las respuestas se embeben porque siempre se muestran junto a la reseña, son pocas y no se consultan por separado. El producto, en cambio, se **referencia** por id porque vive en MySQL y cambia (precio, stock).

Índices: {id_producto: 1, estado: 1, fecha: -1} para listar las reseñas publicadas de un producto; índice **único** {id_producto: 1, autor.email: 1}, que hace cumplir la regla «una reseña por cliente y producto» en la propia base (la API responde 409).


```
{
  "_id": ObjectId("6aeb30500dbf00a6fb8fc822"),
  "id_producto": 1,
  "autor": { "nombre": "Andrés Rivera", "email": "andres@correo.com" },
  "calificacion": 5, "titulo": "Excelente calidad",
  "comentario": "La tela es gruesa y no se transparenta. Después de 5 lavadas sigue igual.",
  "id_variante": 1, "talla": "M", "color": "Negro", "ajuste_talla": "GRANDE",
  "codigo_pedido": "SOL-P-00000001", "compra_verificada": true,
  "atributos": { "calidad_tela": 5, "encoge": false, "igual_a_la_foto": true },
  "estado": "PUBLICADA", "util": 1,
  "respuestas": [ { "id_usuario": 1, "nombre": "Laura Torres",
                    "texto": "¡Gracias Andrés!", "fecha": ISODate("2026-11-04T13:17:04Z") } ],
  "fecha": ISODate("2026-11-04T13:17:04Z")
}
```


7.4 fichas_producto

Campo	Tipo	Oblig.	Descripción
_id	int	Sí	Igual a producto.id_producto (relación 1 a 1 con MySQL).
producto, categoria	string	Sí	Copia descriptiva para filtrar sin consultar MySQL.
atributos	object anidado	Sí	Libre por tipo de prenda; admite subdocumentos y listas (máx. 4 niveles, 16 KB).
cuidados	array<string>	No	Instrucciones de lavado.
etiquetas	array<string>	No	En minúsculas: verano, básicos, denim...
version	int	Sí	Se incrementa ($inc) en cada actualización.
actualizado_por, fecha_actualizacion, fecha_creacion	int, date, date	Sí	Auditoría.

Índices: {categoria: 1} y {etiquetas: 1} (índice multiclave sobre el arreglo). Las búsquedas por atributos usan *collation* en español con *strength: 1*, así «algodon» encuentra «Algodón».

Dos fichas reales de la misma colección, con atributos completamente distintos:

```
{
  "_id": 1, "categoria": "Camisetas",
  "atributos": {
    "material": "Algodón",
    "composicion": [
      { "fibra": "algodón peinado", "porcentaje": 100 }
    ],
    "gramaje_gsm": 240, "cuello": "redondo",
    "manga": "corta", "fit": "oversize",
    "certificaciones": ["OEKO-TEX Standard 100"]
  },
  "etiquetas": ["verano", "básicos"],
  "version": 2
}
```

```
{
  "_id": 2, "categoria": "Jeans",
  "atributos": {
    "material": "Denim",
    "composicion": [
      { "fibra": "algodón", "porcentaje": 98 },
      { "fibra": "elastano", "porcentaje": 2 }
    ],
    "tiro": "alto", "elasticidad": "media",
    "lavado": "stone wash", "bolsillos": 5,
    "cierre": "cremallera YKK"
  },
  "etiquetas": ["denim", "todo el año"],
  "version": 1
}
```

8. Lógica de negocio que vincula ambos tipos de datos

Endpoint	Desde MySQL	Desde MongoDB
GET /api/v1/auth/perfil/actividad	Datos del usuario autenticado: nombre, rol, sede, fecha de registro.	Últimas 10 actividades (?limite= hasta 50) y conteo por acción con $group.
GET /api/v1/usuarios/{id}/actividad	Lo mismo para cualquier usuario (administrador).	Ídem.
GET /api/v1/productos/{id}/completo	Producto, imágenes, variantes, precio final y stock por sede.	Ficha técnica, resumen de reseñas ($facet), 5 reseñas recientes y escaneos QR de 30 días.
GET /api/v1/productos/buscar-atributos	Precio, tallas disponibles y stock de los productos encontrados.	Filtro por etiqueta, categoría o cualquier atributos.<clave>.
GET /api/v1/reportes/qr-mas-escaneados	Descripción, SKU, precio y stock de cada variante del ranking.	Agregación: escaneos por variante y por canal en los últimos N días.
POST /api/v1/productos/{id}/resenas	Verifica que el pedido esté pagado e incluya el producto; copia talla y color.	Inserta la reseña con compra_verificada.

Ejemplo: el endpoint pedido en el enunciado (perfil de usuario desde SQL junto a sus últimas 10 actividades guardadas en NoSQL) se implementa así en *api/rutas/actividad.py*:

```
@bp.get("/auth/perfil/actividad")
@requiere_auth()
def perfil_con_actividad():
    limite = arg_entero("limite", 10, minimo=1, maximo=50)
    usuario = g.usuario                          # cargado de MySQL al validar el JWT
    return jsonify({
        "usuario": usuario_dict(usuario),                              # MySQL
        "ultimas_actividades": actividad.ultimas_actividades(usuario.id_usuario, limite),   # MongoDB
        "resumen_actividad": actividad.resumen_actividad(usuario.id_usuario),              # MongoDB
    })

# servicios/actividad.py
coleccion("actividad_usuario").find({"id_usuario": id_usuario}).sort("fecha", -1).limit(limite)
```

Agregación que alimenta el resumen de reseñas de un producto (una sola consulta a MongoDB):

```
[{ "$match": { "id_producto": 1, "estado": "PUBLICADA" } },
 { "$facet": {
    "general":   [{ "$group": { "_id": null, "total": { "$sum": 1 },
                                "promedio": { "$avg": "$calificacion" },
                                "verificadas": { "$sum": { "$cond": ["$compra_verificada", 1, 0] } } } }],
    "estrellas": [{ "$group": { "_id": "$calificacion", "n": { "$sum": 1 } } }],
    "talla":     [{ "$match": { "ajuste_talla": { "$exists": true } } },
                  { "$group": { "_id": "$ajuste_talla", "n": { "$sum": 1 } } }] } }]
```

9. Operaciones CRUD implementadas

Colección	Create	Read	Update	Delete
fichas_producto	PUT /productos/{id}/ficha (upsert)	GET /productos/{id}/ficha; GET /productos/buscar-atributos	PUT /productos/{id}/ficha ($set + $inc version)	DELETE /productos/{id}/ficha
resenas_producto	POST /productos/{id}/resenas	GET /productos/{id}/resenas (paginado, orden, filtro); GET /resenas/{id}	PATCH /resenas/{id} ($set estado, $push respuesta); POST /resenas/{id}/util ($inc)	DELETE /resenas/{id}
actividad_usuario	Automático en cada petición autenticada	GET /actividad (filtros); GET /auth/perfil/actividad	— (eventos inmutables)	Automático por índice TTL (180 días)
escaneos_qr	Automático en GET /variantes/qr/{codigo}	GET /reportes/qr-mas-escaneados	— (eventos inmutables)	Automático por índice TTL (365 días)

10. Validación y seguridad

- Las claves que empiezan por **$** o contienen **.** se rechazan (422) en atributos libres, para evitar inyección de operadores de MongoDB. Los filtros de búsqueda solo aceptan *etiqueta*, *categoria* y *atributos.<clave>*.
- Límites de tamaño: reseña (2000 caracteres, 15 atributos, 5 fotos), ficha (16 KB, 4 niveles de anidación).
- Los ObjectId mal formados en la URL se tratan como recurso inexistente (404) en lugar de error 500.
- Moderación, eliminación, fichas y bitácora requieren JWT de ADMINISTRADOR; publicar reseñas es público.
- El email del autor se usa para la regla de unicidad pero se excluye (proyección) de toda respuesta pública.

11. Configuración y conexión

Variables nuevas en *.env*: **MONGO_URI** (local *mongodb://localhost:27017* o Atlas *mongodb+srv://...*), **MONGO_DB** y **MONGO_TIMEOUT_MS**. El cliente se crea una sola vez al iniciar la aplicación (*MongoClient* mantiene su propio pool de conexiones) con *serverSelectionTimeoutMS* corto para que una caída de MongoDB no bloquee la API. Los errores de PyMongo se traducen a una respuesta JSON 503 uniforme.

12. Interfaz web y pruebas automáticas

Aunque el enunciado se centra en la API, todos los datos NoSQL se pueden ver y operar desde una interfaz web servida por la misma aplicación Flask:

- **Tienda para clientes** (*/*, */producto*, */escanear*, */carrito*): la ficha técnica se dibuja como la etiqueta cosida de la prenda, las opiniones muestran el resumen calculado con *$facet* y cada lectura de un QR queda en *escaneos_qr*.
- **Panel del personal** (*/panel*): editor de fichas técnicas con vista previa, moderación de opiniones, bitácora de actividad, reporte de prendas más escaneadas y la sección «Mi perfil», que consume el endpoint combinado */auth/perfil/actividad*.

El repositorio incluye **47 pruebas automáticas** con pytest que se ejecutan contra MySQL y MongoDB reales, en bases aparte (*solsticio_test*) para no tocar los datos de desarrollo. Cubren el CRUD de fichas y reseñas, la compra verificada contra MySQL, los endpoints combinados, la bitácora, los escaneos y el comportamiento de la API cuando MongoDB no está disponible.

13. Evidencias de ejecución

Capturas tomadas en un equipo local con MySQL 8.0 y MongoDB 9.0, con el catálogo de ejemplo cargado mediante *flask --app app sembrar-demo*. El README del repositorio contiene además las respuestas de la API para cada endpoint.

```
PS C:\Uni\solsticio> flask --app app verificar-bd
Conexión correcta. Tablas en la base de datos: 23

PS C:\Uni\solsticio> flask --app app verificar-mongo
Conexión correcta a MongoDB 9.0.4, base 'solsticio'.
  actividad_usuario: 100 documentos
  escaneos_qr: 16 documentos
  resenas_producto: 13 documentos
  fichas_producto: 12 documentos

PS C:\Uni\solsticio> pytest
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0
tests\test_combinados.py .........                                       [ 21%]
tests\test_degradacion.py ...                                            [ 27%]
tests\test_fichas.py .......                                             [ 42%]
tests\test_panel_api.py .....                                            [ 53%]
tests\test_resenas.py ................                                   [ 89%]
tests\test_web.py .....                                                  [100%]
============================== 47 passed in 6.95s ==============================
```

Figura 3. Conexión a MySQL y MongoDB, documentos por colección y las 47 pruebas automáticas aprobadas.

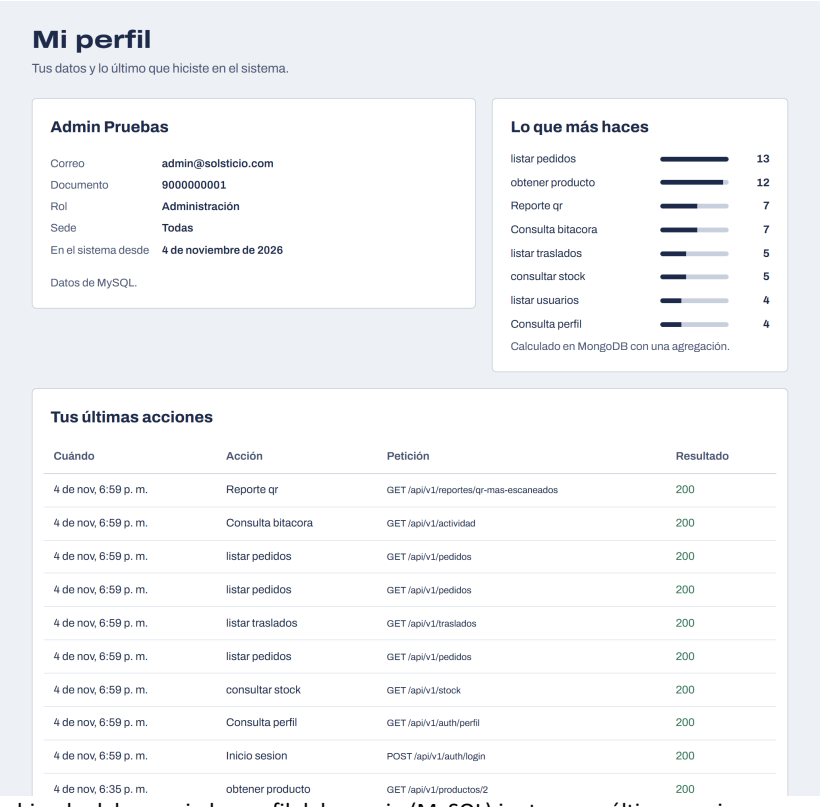


Figura 4. Endpoint combinado del enunciado: perfil del usuario (MySQL) junto a sus últimas acciones y su resumen (MongoDB).

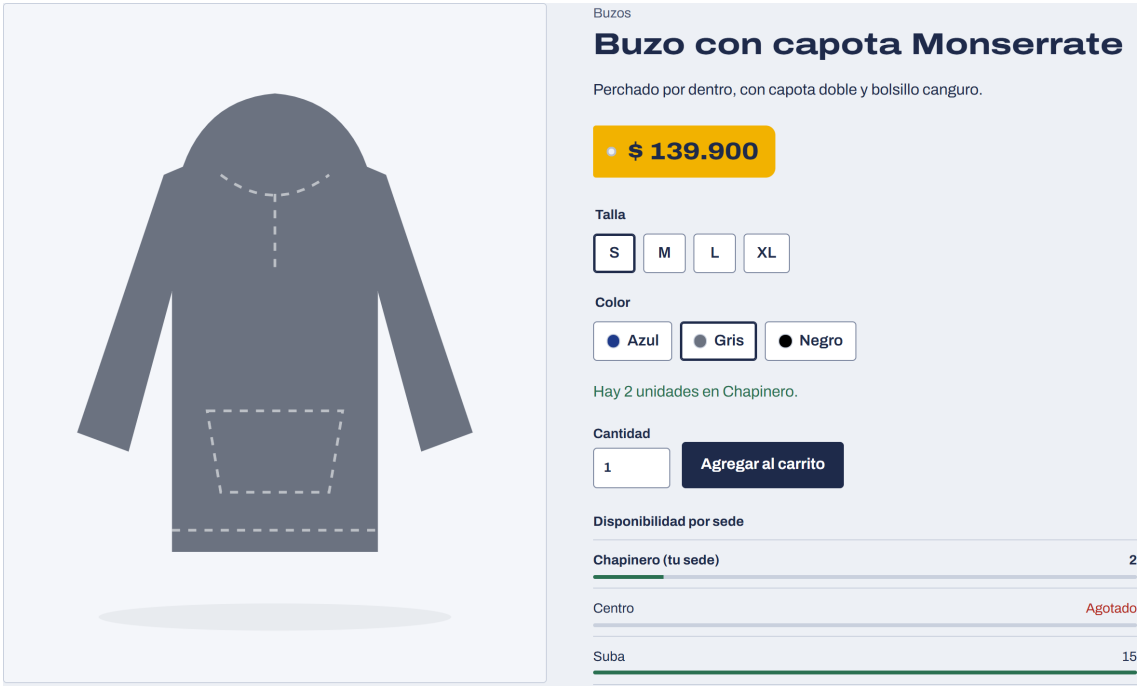


Figura 5. Detalle de una prenda: precio, tallas y existencias por sede leídos de MySQL.

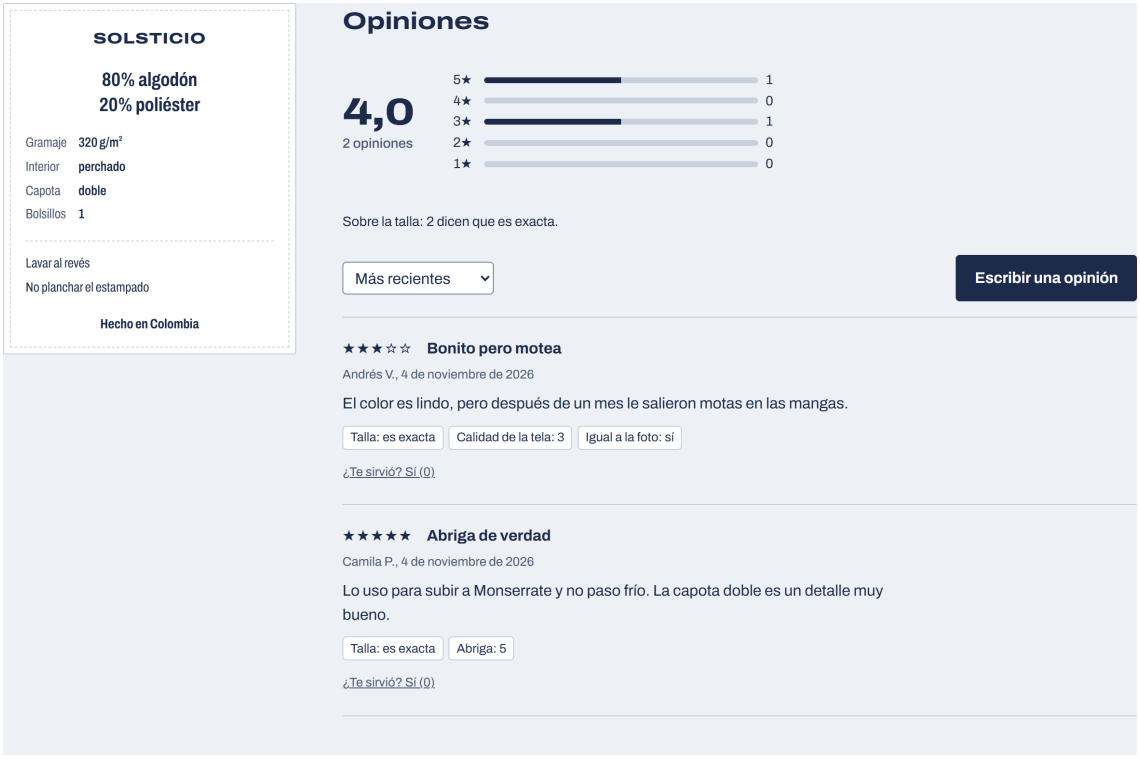


Figura 6. La misma prenda: ficha técnica (fichas_producto) y opiniones con su resumen (resenas_producto), desde MongoDB.



Figura 7. Reporte de prendas más escaneadas: agregación sobre escaneos_qr completada con precio y existencias de MySQL.

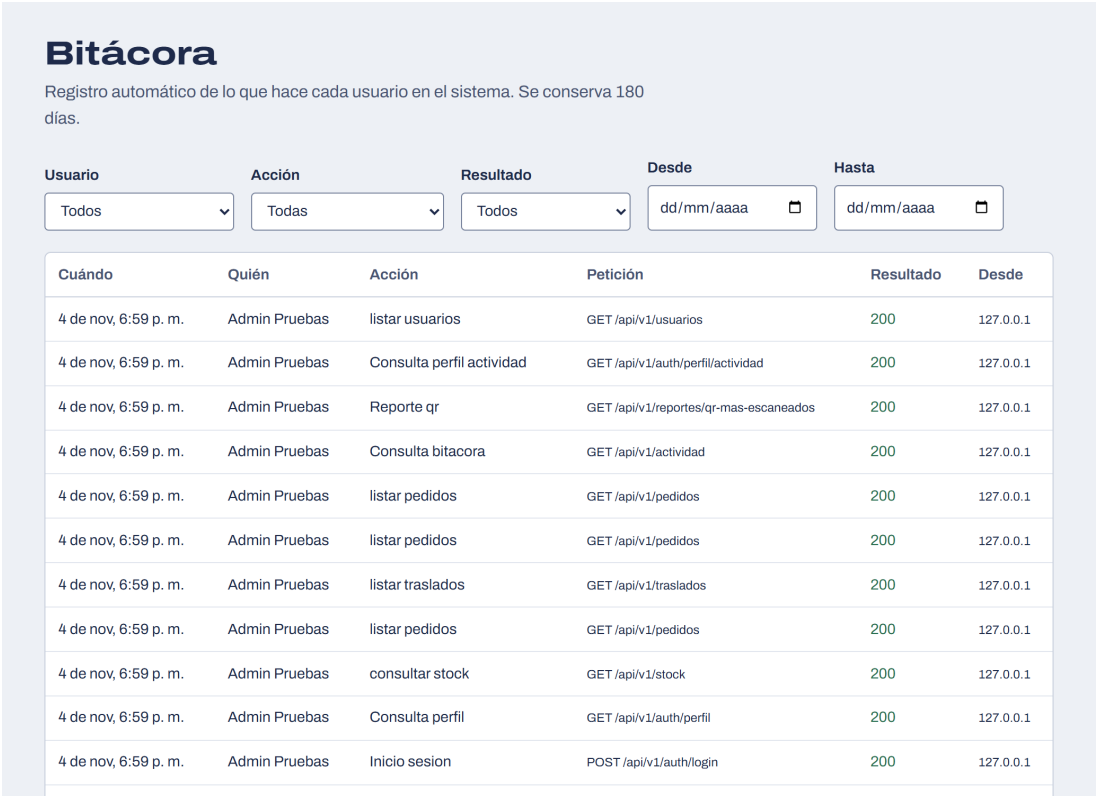


Figura 8. Bitácora de actividad (actividad_usuario) con filtros por usuario, acción, resultado y fechas.

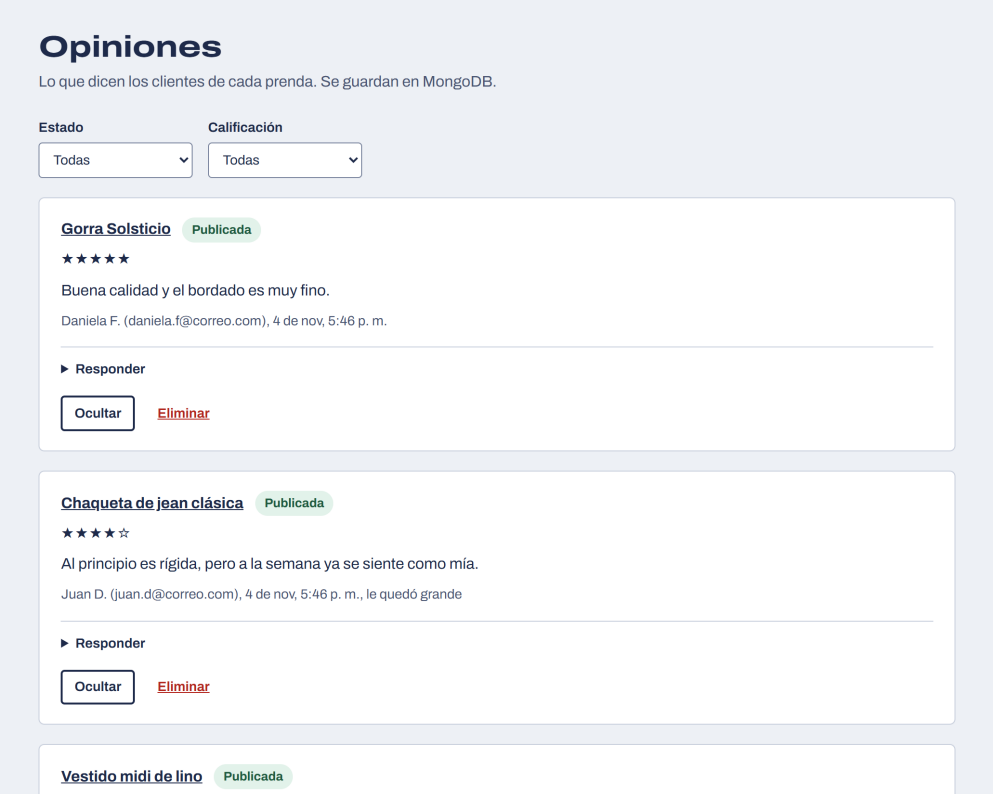


Figura 9. Moderación de opiniones: responder ($push), ocultar o publicar ($set) y eliminar.

14. Conclusiones

- MySQL sigue garantizando la integridad de lo que mueve dinero y mercancía; MongoDB absorbe los datos variables y de alta frecuencia sin cargar las tablas transaccionales ni exigir migraciones de esquema.
- El enlace por identificadores y la regla «precio y stock solo en MySQL» evitan inconsistencias entre ambas bases.
- Los índices TTL resuelven la retención de eventos sin tareas programadas, y el pipeline de agregación permite reportes (ranking de QR, resumen de reseñas) directamente en la base.
- La aplicación degrada con elegancia: una caída de MongoDB no detiene las ventas.

Referencias

MongoDB, Inc. (s. f.). *Data modeling*. MongoDB Manual. Recuperado el 4 de noviembre de 2026, de https://www.mongodb.com/docs/manual/data-modeling/

MongoDB, Inc. (s. f.). *Aggregation pipeline*. MongoDB Manual. Recuperado el 4 de noviembre de 2026, de https://www.mongodb.com/docs/manual/core/aggregation-pipeline/

MongoDB, Inc. (s. f.). *TTL indexes*. MongoDB Manual. Recuperado el 4 de noviembre de 2026, de https://www.mongodb.com/docs/manual/core/index-ttl/

MongoDB, Inc. (s. f.). *PyMongo documentation*. Recuperado el 4 de noviembre de 2026, de https://pymongo.readthedocs.io/

Oracle Corporation. (s. f.). *MySQL 8.0 reference manual*. Recuperado el 4 de noviembre de 2026, de https://dev.mysql.com/doc/refman/8.0/en/

Pallets Projects. (s. f.). *Flask documentation*. Recuperado el 4 de noviembre de 2026, de https://flask.palletsprojects.com/

Sadalage, P. J., y Fowler, M. (2012). *NoSQL distilled: A brief guide to the emerging world of polyglot persistence*. Addison-Wesley.

SQLAlchemy. (s. f.). *SQLAlchemy 2.0 documentation*. Recuperado el 4 de noviembre de 2026, de https://docs.sqlalchemy.org/en/20/